Interactive MTObjects Tester: Code Process and Flow (No Spike Gate)

Implementation walkthrough for maintaining and extending the tester after spike-gate removal.

Design intent: MTObjects is isolated behind a product-producing adapter so SEP, MTObjects, and future detectors can be compared through common masks, rows, profiles, and PNG outputs. The tester now runs exactly one detector pass per galaxy.

Runtime Entry Point

1. parse_args reads --manifest, --pc, and --mtobjects-root.
2. main creates MTObjectsTester with the selected manifest, machine, and MTObjects checkout path.
3. MTObjectsTester loads the local manifest through interactive_galclean_parameter_tester helpers.
4. The Tkinter UI exposes machine, galaxy, detection surface, units, MTObjects parameters, and post-filters; every slider/spinbox label states which direction is more aggressive.

Calculation Flow

Stage	Function	Responsibility
Load FITS	load_fits / _load_galaxy	Read image data and geometry, with a per-galaxy cache.
Detection image	prepare_detection_image	Create original or smoothed-residual detection image and non-finite mask.
MTObjects setup	mtobjects_context	Temporarily switch into the MTObjects checkout so its compiled libraries load correctly.
Parameter object	mtobjects_parameter_namespace	Build the SimpleNamespace expected by MTObjects internals.
Tree detection	mtobjects_products	Preprocess image, build max tree, filter tree, relabel segments, and produce raw rows.
Post-filtering	measure_segments / filter_segmentation	Measure labels and apply min/max area, elongation, centre exclusion, and dilation.
Visual output	draw_products	Render original, residual, isophotes, and profiles; save a PNG.

Product Contract

The single MTObjects run returns a dictionary shaped like the SEP product boundary. This keeps downstream drawing and future benchmark code simple.

Key	Type	Meaning
raw_segmentation	2D int array	Relabeled MTObjects segmentation before comparison-layer filtering.
filtered_segmentation	2D int array	Segmentation after geometry and size filters.
mask	2D bool array	Dilated foreground mask used for visual/profile cleaning.
cleaned	2D float array	Original image with mask pixels replaced by the finite unmasked median.
residual	2D float array	Original minus smoothed model, used for display and measurements.
rows	list[dict]	Per-label measurements and keep/reject state.
background_level / background_rms	float	MTObjects estimated or supplied background values for audit display.

What Changed From the Spike-Gate Version

- Removed functions: spike_gated_mtobjects_products, detect_profile_spikes, spike_samples_to_image_aperture, expand_boolean_mask.
- Removed constants: SPIKE_GATE_MOVE_FACTOR and the seven DEFAULT_SPIKE_* defaults.
- Removed GUI: the 'Spike Gate' LabelFrame and its seven controls.
- Removed figure panel: the third (spike-gated) profile axis; profile_grid is now 2 rows instead of 3.
- draw_profile no longer takes a spike_samples argument; its legend now shows whenever a bridged (log-linear interpolated) profile is drawn, rather than only on the removed spike-gated panel.
- MTObjects/Post-filter control labels gained a '(↑/↓ more aggressive)' suffix documenting which direction increases masking; direction is not shown for parameters where the effect is not simply

more/less aggressive (gaussian_fwhm) or that characterise the noise model rather than detection strictness (soft_bias, gain, bg_mean, bg_variance).
- Output folder renamed from interactive_mtobjects_parameter_tester to interactive_mtobjects_parameter_tester_no_spike_gate.
- mtobjects_toy_object_parameter_optimisation.py's default_params() no longer sets the seven spike_* keys, which were always unused there (only mtobjects_products was ever called, never spike_gated_mtobjects_products).

Error Handling and Setup

- If MTObjects cannot import or load its shared libraries, the adapter raises a setup-focused error message.
- Non-finite pixels are replaced only for MTObjects preprocessing, then restored to background in the segmentation.
- The max-tree C memory is released in a finally block after filtering.
- The script syntax-checks without MTObjects installed because imports occur inside the runtime adapter.

Maintenance Notes

- For a formal SEP versus MTObjects benchmark, call detector product functions directly and compare mask fraction, object counts, profile residuals, runtime, and manual review flags.
- Keep new detector adapters returning the same product contract rather than branching the plotting code.
- If MTObjects is packaged differently later, update mtobjects_context only; the rest of the tester should not need to change.
- If replacement semantics change from median fill to interpolation/inpainting, implement that after product creation so detectors remain comparable.
- If spike-gating is needed again, interactive_sep_spike_gate_parameter_tester.py still implements the equivalent idea for SEP and can be used as a reference.

Systematic documentation review - 17 August 2026

Review classification: Reviewed - code-flow reference remains valid

Updates made: No stale zero-mask conclusion, six-panel batch claim or obsolete optimisation result was found. A scope note was added.

Current interpretation: The code-flow description remains useful for maintenance; final Toy Objects parameter selection and batch status are maintained separately.

Authoritative combined outcome: SEP and MTObjects Masking Comparison and Recommendations 20260817.docx. Full optimisation history and final science-image SEP fold records: Foreground Masking Optimisation Results.xlsx.